Problem A. Special Edges

Time limit 5000 ms
Mem limit 262144 kB

Koa the Koala has a **directed** graph G with n nodes and m edges. Each edge has a capacity associated with it. Exactly k edges of the graph, numbered from 1 to k, are special, such edges initially have a capacity equal to 0.

Koa asks you q queries. In each query she gives you k integers $w_1, w_2, \ldots, w_k$. This means that capacity of the i-th special edge becomes w_i (and other capacities remain the same).

Koa wonders: what is the maximum flow that goes from node 1 to node n after each such query?

Help her!

Input

The first line of the input contains four integers n, m, k, q ($2 \le n \le 10^4$, $1 \le m \le 10^4$, $1 \le k \le \min(10, m)$, $1 \le q \le 2 \cdot 10^5$) — the number of nodes, the number of edges, the number of special edges and the number of queries.

Each of the next m lines contains three integers u, v, w ($1 \le u, v \le n$; $0 \le w \le 25$) — the description of a directed edge from node u to node v with capacity w.

Edges are numbered starting from 1 in the same order they are listed in the input. The first k edges are the special edges. It is guaranteed that $w_i = 0$ for all i with $1 \le i \le k$.

Each of the next q lines contains k integers $w_1, w_2, \ldots, w_k$ ($0 \le w_i \le 25$) — the description of the query. w_i denotes the capacity of i-th edge.

Output

For the i-th query, print one integer res_i — the maximum flow that can be obtained from node 1 to node n given the i-th query's special edge weights.

Sample 1

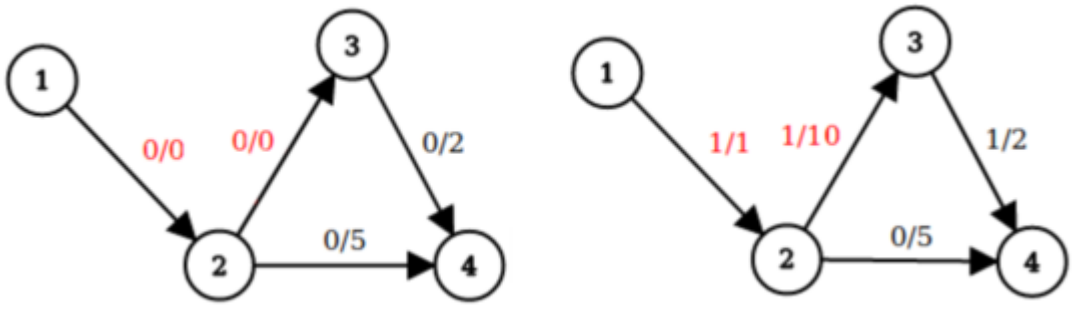
Input	Output
2 1 1 3 1 2 0 0 1 2	0 1 2

Sample 2

Input	Output
4 4 2 5 1 2 0 2 3 0 2 4 5 3 4 2 0 0 1 10 10 0 7 1 7 2	0 1 5 6 7

Note

For the second sample, the following images correspond to the first two queries (from left to right respectively). For each edge there is a pair `flow/capacity` denoting flow pushed through the edge and edge's capacity. The special edges are colored in red.



As you can see in first query maximum flow from node 1 to node 4 equals 0 and in second query equals 1.

Problem B. Eroding Pillars

Time limit	6000 ms
Mem limit	1048576 kB
OS	Linux

You've landed yourself in a real mess, or more accurately a cave filled with dilapidated and unstable pillars. You are luckily standing on a solid rock in the middle of the cave. You know there is a valuable artifact on one of the pillars in the cave, but you aren't sure which one yet. While you wait for the results of a scan of the cave, you start building a robot to help you retrieve the artifact.

The robot you build will be light, and it will assuredly be able to land and jump from any pillar at least once. To build the robot, you need to determine how powerful it needs to be in terms of its jump distance. If the robot is too weak, then it might not be able to reach a pillar. If the robot is too strong, then it will cause a lot of damage when jumping and landing.

Luckily you have already mapped out where all the pillars are located with respect to your starting point at $(0, 0)$. You don't know yet which pillar contains the artifact, and you need to finish building the robot first before the scan finishes.

Given the locations of the pillars, you'd like to determine the smallest jump distance to guarantee that your robot can reach any pillar and return back to the start, without landing on a pillar twice.

Input

The first line of input contains a single integer n $(1 \leq n \leq 1,000)$, which is the number of pillars.

Each of the next n lines contains two integers, x and y $(-10^9 \leq x, y \leq 10^9)$. These are the ($x$,$y$) coordinates of the pillars. All pillar locations will be distinct, and no pillar will be at $(0, 0)$.

Output

Output a single number, which is the minimum jump distance needed to guarantee that a robot could reach any pillar and return back to your starting point, without landing on a pillar twice. Your answer will be accepted if it has an absolute or relative error of at most 10^{-6}.

Sample 1

Input	Output
2 1 1 1 0	1.414213562373095

Sample 2

Input	Output
8 1 1 0 1 1 0 2 0 0 2 2 1 1 2 2 2	1.0

Problem C. AND Permutation

Time limit	2000 ms
Mem limit	2097152 kB
OS	Linux

You are given a sequence of n distinct nonnegative integers $a_1, a_2, \ldots, a_n$.

For the given sequence, it is guaranteed that for all nonnegative numbers x, if there is some i such that $a_i \ \& \ x = x$, then there is a j such that $a_j = x$. Here, $\&$ refers to the bitwise *AND* operator.

Find a permutation $b_1, b_2, \ldots, b_n$ of $a_1, a_2, \ldots, a_n$ such that $b_i \ \& \ a_i = 0$ for all i. If there are multiple solutions, find any such permutation. It is guaranteed that a solution always exists.

Input

The first line of input contains an integer n ($1 \leq n < 2^{18}$), which is the number of integers in the permutation.

Each of the next n lines contains an integer a_i ($0 \leq a_i < 2^{60}$), which is the input sequence, in order of i. All of the a_i's are guaranteed to be distinct. For all nonnegative numbers x, if there is some i such that $a_i \ \& \ x = x$, then there is a j such that $a_j = x$.

Output

Output n lines, each containing a single integer, which are the b_i's, in order of i.

Sample 1

Input	Output
6 0 1 4 5 2 6	4 6 0 2 5 1

Problem D. Exponent Exchange

Time limit	1000 ms
Mem limit	1048576 kB
OS	Linux

Alice and Bob are playing a cooperative game. They hold b^p dollars between them, for given integers b and p. Alice initially holds x dollars, and Bob holds $b^p - x$ dollars. Alice and Bob want to consolidate their money, so one person holds all the money.

In each transaction, one player can choose to send the other player b^y dollars, for some integer y with $0 \le y < p$. But each player wants to initiate as few transactions as possible. They are willing to cooperate such that the player that initiates the most transactions (the busiest player), initiates as few as possible.

Alice and Bob want to know the fewest number of transactions that the busiest player needs to initiate to complete the transfer.

Input

The first line of input contains two integers b ($2 \le b \le 100$) and p ($2 \le p \le 1000$), where b is the base, and p is the number of digits.

The next line contains p integers $x_{p-1}, x_{p-2}, \ldots, x_0$, separated by spaces, with $0 \le x_i < b$ and $0 < x_{p-1}$. These are the base-b digits of the value of x, with the most significant digit first. Specifically, $x = \sum_{0 \le i < p} b^i x_i$. Note that they are given in order from the highest power to the lowest. For example, in the sample, `4 2 7 8 6` with $b = 10$ represents the base 10 number 42 786.

Output

Output a single integer, which is the minimum number of transactions the busiest player must initiate to transfer all the money to either Alice or Bob.

Sample 1

Input	Output
`10 5` `4 2 7 8 6`	`7`

Problem E. Drying Laundry

Time limit	3000 ms
Mem limit	262144 kB
OS	Windows

Harry the Beaver runs a hotel and has to wash bed sheets every Sunday night for the next Q weeks until the tourist season ends. On week j, he has N freshly washed bed sheets that he wants to dry by hanging them on two parallel clotheslines of length L_j each. The sheets can be hung next to each other but must not overlap. Each sheet is di units wide and rather long, therefore he will always orient it so that it will take up d_i units of the line when hung to dry. The sheets have different drying times that are not related to their sizes because of different materials. Thus, the i-th sheet needs t_i^{slow} minutes to dry. However, if it is hung over both lines at the same time, it dries quicker in t_i^{fast} minutes, but also takes up space on the other line. To avoid smelly sheets, Harry the Beaver has to start drying all of them immediately after washing, i.e. all sheets have to be hung simultaneously.

Harry the Beaver wants to go to sleep as soon as possible on Sundays, therefore, he asks you to help him determine the minimal required drying time for each week j, or inform him that it is impossible to finish drying the sheets that week.

Input limits

- $1 \le N \le 3 \cdot 10^4$
- $1 \le Q \le 3 \cdot 10^5$
- $1 \le d_i \le 3 \cdot 10^5$ for all $1 \le i \le N$
- $1 \le t_i^{\text{fast}} \le t_i^{\text{slow}} \le 10^9$ for all $1 \le i \le N$
- $1 \le L_j \le 3 \cdot 10^5$ for all $1 \le j \le Q$

Input

The first line contains an integer N, the number of sheets, and an integer Q, the number of weeks until the end of the tourist season. The next N lines contain space-separated integers d_i, t_i^{fast}, and t_i^{slow}, which correspond to the width, the shorter drying time, and the longer drying time of the i-th sheet, respectively. The final Q lines the the input contain integers L_j, j-th of which represents the length of the clothesline for week j.

Output

Print Q lines, with j-th of them containing the minimal required drying time for week j, or `"-1"` (without the quotes) if it is impossible to finish drying the sheets that week.

Sample 1

Input	Output
3 3 1 2 2 1 1 4 2 3 100 3 1 4	4 -1 3

Problem F. Token Game

Time limit	3000 ms
Mem limit	2097152 kB
OS	Linux

Alice and Bob are playing a game on board which is a 2-dimensional 300×300 grid. The board is subdivided into cells. Each cell can be uniquely identified by two integers representing (x, y) coordinates, each in the range from 1 to 300.

Two tokens are on the board on distinct cells. Alice starts the game. On each player's turn, that player chooses one of the tokens, chooses one of the coordinates of the cell it's on, and reduces that coordinate by some positive amount. The moved token cannot jump over or occupy the same space as the other token. The token must also remain on the board (so both of its coordinates need to stay positive). The first player unable to make a move loses. Note that both players can move either token.

You are given the starting configuration of a number of games. For each of the games, compute the number of initial winning moves available to Alice.

Input

The first line of input contains a single integer n ($1 \le n \le 10^5$), which is the number of games to analyze.

Each of the next n lines contains four integers x_1, y_1, x_2 and y_2 ($1 \le x_1, x_2, y_1, y_2 \le 300$, and either $x_1 \ne x_2$ or $y_1 \ne y_2$ holds). This represents the starting configuration of one game, with the tokens at cells (x_1, y_1) and (x_2, y_2).

Output

Output n lines. On each line, output a single integer, which is the number of initial winning moves available to Alice for one of the input games. Output them in the order of the input.

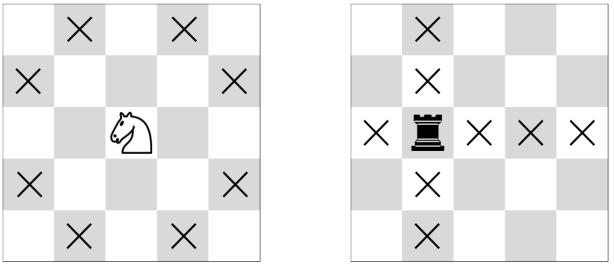
Sample 1

Input	Output
5 6 6 6 3 6 6 2 2 1 6 3 1 3 6 1 3 6 3 1 5	3 0 1 1 0

Problem G. Lone Knight

Time limit	7000 ms
Mem limit	1048576 kB
OS	Linux

In the game of chess, a knight moves as shown in the picture below; each move is one square horizontally and two squares vertically or two squares horizontally and one square vertically. A rook can move any number of squares horizontally or vertically, but not both in the same move. If a square can be reached by a rook in one move, that square is said to be attacked by the rook.



Consider an infinite chess board, with squares that can be indexed by integer coordinates. There is a white knight on the board on a square, and it wants to go to another square. However, there are also a number of black rooks on the board. The knight can make as many moves as it needs to get to its target square, but it cannot stop on a square that is attacked by or occupied by a rook. The rooks don't move.

Can the white knight reach its target square? You are to answer that question many times!

Input

The first line of input contains two integers n and q ($1 \le n, q \le 1\,000$), where n is the number of black rooks and q is the number of queries.

Each of the next n lines contains two integers x and y. This indicates that there is a black rook at (x, y). No two rooks share the same square.

Each of the next q lines contains four integers x_s, y_s, x_t and y_t. This is a query, where the white knight starts at square (x_s, y_s) and wants to move to square (x_t, y_t).

All square coordinates in the input are no larger than 10^9 in absolute value. It is guaranteed that in every query the knight's initial and target squares are not attacked by or occupied by any rook, and the target square is not the same as the initial square.

Output

For each query, output on a single line `1` if the knight can reach the target square, or `0` otherwise.

Sample 1

Input	Output
6 6 10 14 1 0 0 1 4 9 9 13 5 9 2 2 3 4 2 2 2 4 2 2 6 4 2 2 2 10 7 11 6 2 6 2 8 12	1 0 0 1 0 1

Sample 2

Input	Output
8 10 0 0 1 1 5 5 8 8 11 11 14 14 18 18 19 19 17 10 13 9 15 15 15 9 7 3 17 4 15 15 12 3 9 17 3 3 4 4 9 4 12 12 2 6 10 15 6 6 15 17 4 16 -1000000000 -999999999 15 7	1 0 0 0 0 0 0 0 0 0

Problem H. Jumbled Stacks

Time limit	1000 ms
Mem limit	262144 kB
OS	Windows

We are given a set of n cards, labelled from 1 to n, which are distributed into k stacks $S_1, S2, \ldots, S_k$. Each stack has a limited capacity: the i-th stack, S_i, can contain at most C_i cards. The only way we can manipulate these cards is by taking the top card of a stack and moving it to the top of some other stack (as long as this wouldn't exceed the capacity of the destination stack).

Using a sequence of such moves, we would like to rearrange the cards so that the first few stacks (0 or more) with the smallest indices are filled to capacity, the stack immediately after them is not filled to capacity (and may even be empty) and all stacks after that are completely empty. Moreover, if we stack together all the stacks from S_1 at the bottom to S_k at the top, the cards should be ordered from smallest to largest, with 1 at the bottom and n at the top.

It is guaranteed that $n \leq \left(\sum_{i=1}^{k} C_i\right) - \max_{1 \leq i \leq k} C_i$.

Suppose we had $n = 6$ cards on $k = 3$ stacks, with capacities $C_1 = 4, C_2 = C_3 = 3$, and with the following initial state: $S_1 = [2, 3, 0, 0]$ (from bottom to top; 0 indicates an empty slot), $S_2 = [4, 1, 6]$, $S_3 = [5, 0, 0]$. Then the desired end state is $S_1 = [1, 2, 3, 4]$, $S_2 = [5, 6, 0]$ and $S_3 = [0, 0, 0]$.

Input

The first line contains two integers, n (the number of cards) and k (the number of stacks), separated by a space. The remaining k lines describe the initial state of the stacks; the i-th of these lines describes S_i and contains C_{i+1} integers, separated by spaces. The first of these integers is C_i (the capacity of the stack S_i), the rest of them are the labels of the cards on S_i, from bottom to top. If the stack Si contains fewer than C_i cards (it could even be empty), the last few integers in the line will be 0.

Input limits

- $1 \leq n \leq 100$
- $3 \leq k \leq 100$

- $1 \leq C_i \leq n$

Output

Print a sequence of moves that bring the stacks into the desired end state. For each move, output a line containing two integers, separated by a space: first the number of the stack from which the card is being moved and then the number of the stack to which it is being moved (the stacks are numbered from 1 to k; the destination stack must not be the same as the source stack). The number of moves must not exceed 10^5. After the end of the sequence of moves, print a line containing "`0 0`" (without the quotation marks). If there are several possible solutions, you may output any of them.

Sample 1

Input	Output
6 3 4 2 3 0 0 3 4 1 6 3 5 0 0	2 3 2 3 1 2 1 2 3 1 2 1 2 1 3 2 3 1 2 3 1 3 2 1 3 2 3 2 0 0

Note

This is the example discussed earlier in the problem statement. The sample output shows a sequence of 14 moves which bring the stacks into the desired end state.

Problem I. Ketek Counting

Time limit	4000 ms
Mem limit	65536 kB
OS	Linux

Define a *Ketek* to be a sentence that reads the same forwards and backwards, by word. For example, ‘fall leaves after leaves fall’ is a *Ketek* since the words in reverse order are the same as the original order.

Given a string consisting of lower-case letters and the character ‘`?`’, count the number of distinct *Keteks* you can make by replacing every ‘`?`’ with lower-case letters (one letter per ‘`?`’), and optionally adding spaces between any letters. Note that a *Ketek* cannot contain any `?`’s; they all must be replaced exclusively by lower-case letters.

For example, if we start with the string ‘`ababa`’, we can form 3 different *Keteks*: ‘`ababa`’, ‘`a bab a`’ and ‘`a b a b a`’.

If we start with the string ‘`?x?z`’ instead, we can form 703 different *Keteks*:

- There are $26^2 = 676$ ways to replace the `?`’s and form a one-word *Ketek*.
- Add spaces to form ‘`? x? z`’. There are 26 ways to form a *Ketek* (the first ‘`?`’ must be `z`; the other can be any lower-case letter).
- Add a space to form ‘`?x ?z`’. There is no way to form a *Ketek*.
- Add spaces to form ‘`? x ? z`’. There is one way to form a *Ketek* (the first ‘`?`’ must be `z`; the second must be `x`).

The total is $676 + 26 + 0 + 1 = 703$.

Two *Keteks* are different if they have a different number of words, or there is some word index where the words are not the same.

Input

The single line of input contains a string s ($1 \le |s| \le 30\,000$), which consists of lower-case letters (‘`a`’–‘`z`’) and the character ‘`?`’.

Output

Output the number of distinct *Keteks* that can be formed by replacing the ?'s with lowercase letters and adding spaces. Since this number may be large, output it modulo $998\,244\,353$.

Sample 1

Input	Output
`ababa`	`3`

Sample 2

Input	Output
`?x?z`	`703`

Problem J. Triangle Containment

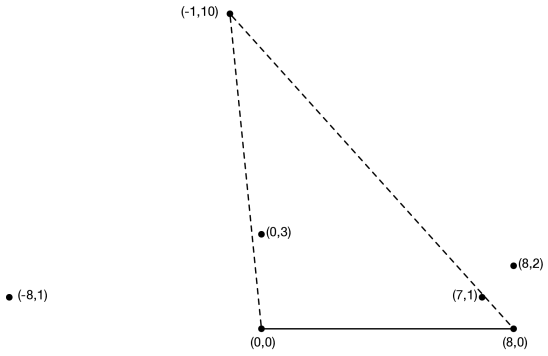
Time limit	3000 ms
Mem limit	1048576 kB
OS	Linux

You recently discovered there is treasure buried on your farm land. A **lot** of treasure! You quickly decide to put a fence around the land.

Alas, you have but a single fence post! You will have to drive to town to get more fencing material. But you can't just leave the land as open as it is, so you decide to create a makeshift fence to protect some of the treasure while you are gone. You will place the post in the ground and run some wire in a straight line between two sides of your barn wall and the fence post to section off a triangular area. Also, the ground is very hard: only places that were dug up to bury a treasure are soft enough for you to quickly place the fence post.

To figure out the best option, you first calculate the following. For each of the treasures in your field, if you were to place the fence post at that treasure and complete the fence as described, then what is the total value of all treasures that would be enclosed by the fence? Note that the treasure under the post you place is not considered enclosed by the fence (it might not be safe since someone could dig around the post).

Sample Input 1 is illustrated below. The triangle that includes the point $(-1, 10)$ encloses exactly two other treasure points which have total value $4 + 8 = 12$.



Input

The first line of input contains two integers n ($1 \le n \le 10^5$) and x ($1 \le x \le 10^9$), where n is the number of treasure points and x fixes the two corners of the barn wall at locations $(0, 0)$ and $(x, 0)$.

Each of the next n lines contains three integers x, y, and v ($-10^9 \le x \le 10^9$, $1 \le y \le 10^9$, and $1 \le v \le 10^9$) giving the location (x, y) and value v of one of the treasure points. All of

these points are distinct. It is also guaranteed that for each point, the triangle formed with the barn wall will not contain any other treasure point on its boundary.

Output

Output n lines, one for each treasure point in the order of the input. For each point output a single integer, which is the total value of all points in the interior of the triangle that point forms with the barn wall. Note that the value of the point itself should be excluded from this sum.

Sample 1

Input	Output
5 8 -8 1 1 -1 10 2 0 3 4 7 1 8 8 2 16	0 12 0 0 8

Sample 2

Input	Output
6 6 0 1 1 2 3 10 2 5 100 3 1 1000 3 5 10000 4 5 100000	0 1000 1010 0 1010 1000

Problem K. Keys

Time limit	4000 ms
Mem limit	262144 kB
OS	Windows

Alice and Bob live in a massive mansion with n rooms (one of them denoting outdoors, where they play the Moon game) and m doors between them. Each door connects two rooms or a room with the outdoors and has a single unique key that opens this door only. Every door closes behind you and locks automatically, so one always needs a key to pass through. The building is very large, but Alice and Bob only use one room – their bedroom. Other rooms are only there to make the house look bigger and make the neighbors jealous.

This unusual way of building their house is now causing some trouble for Alice and Bob. Bob is leaving for a two-week-long trip. In a week, Alice is also going abroad for a month and when she leaves, she needs the right keys to leave the house. However, Bob also needs keys to get back in since Alice will not be there at the time to let him in. Alice and Bob are now trying to figure out how to split the keys to the doors for Alice to be able to get from room 0 (their bedroom) to 1 (outdoors) and Bob to go from room 1 (outdoors) to 0 (bedroom) one week later.

Fortunately, Alice remembered that she could drop some of the keys on her way out for Bob to pick up on his way back. This way, they can both pass through the same doors. She, of course, cannot drop any keys in room 1 (outdoors) since the neighbors could find them and break into their house.

Can you help Alice and Bob split their keys and plan their trip through the house?

Task

You are given the description of Alice and Bob's mansion: m doors between n rooms numbered 0 to $n-1$, where 1 is outdoors and 0 is the bedroom. The i-th door can be opened with the key number i (0-indexed).

You should first print two lines containing space-separated numbers of keys for Alice and Bob, respectively. It is fine if they do not use all the keys, but it is not allowed for them to both have a copy of the same key (or for one of them to have multiple copies of a key).

You should then print instructions that Alice and Bob will follow. First, print Alice's movements from room 0 to 1 by printing commands of one of the two types:

- "`MOVE x`" to move to room x (assuming that there is a door between Alice's current location and x and that Alice has the key),
- "`DROP k1 k2 ...`" to drop keys $k_1, k_2, \ldots$ (printed as space-separated integers) in the room where Alice currently stands. This means that Alice no longer carries these keys.

Once Alice is done with her movements, print "`DONE`" in a new line. She should finish her movement in room 1. It is fine if Alice passes through room 0 or 1 multiple times while following the printed instructions.

Second, print Bob's movements from room 1 to 0 by printing commands of one of the two types:

- "`MOVE x`" to move to room x (assuming that there is a door between Bob's current location and x and that Bob has the key),
- "`GRAB`" to grab any keys in the room where Bob is currently standing. Bob always grabs all the keys that Alice left in the room. If Alice left none, he does not grab any.

Once Bob is done with his movements, print "`DONE`" in a new line. He should finish his movement in room 0. It is fine if Bob passes through room 0 or 1 multiple times while following the printed instructions.

Remarks: It is considered acceptable, albeit useless, to `DROP` an empty list of keys or to grab keys in a room that has no keys, or grab keys in room 1 (i.e. outside).

Input

The first line contains integers n and m, the number of rooms (outdoors included) and number of doors. This is followed by m lines that describe the doors. The i-th line (count starting from 0) is described by integers a_i, b_i, indicating that there is a door between rooms a_i and b_i, opened with key i.

Input and output limits

- $2 \leq n, m \leq 10^5$
- $0 \leq a_i, b_i < n$
- It is guaranteed that it's always possible to reach any room from any other room if you have all the keys.
- Each pair of rooms is connected with at most one door.
- No room is connected to itself.

- Your program may print at most $4 \cdot 10^5$ instructions.

Output

First, print two lines, describing the split of keys. Then, print all the instructions for Alice and Bob, as described in the task description, one per line. If there is no solution, print "`No solution`" (without the quotes). If there are multiple valid solutions, any of them is accepted.

Sample 1

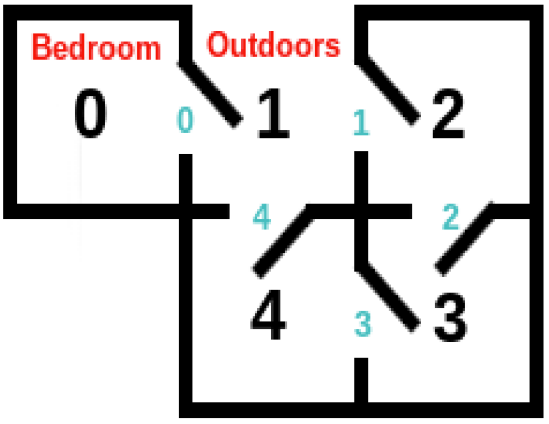
Input	Output
5 5 0 1 1 2 2 3 3 4 4 1	0 1 2 3 4 MOVE 1 MOVE 2 MOVE 3 DROP 0 MOVE 2 MOVE 1 DONE MOVE 4 MOVE 3 GRAB MOVE 4 MOVE 1 MOVE 0 DONE

Sample 2

Input	Output
3 2 0 2 1 2	No solution

Note

The first example represents the following floor plan, where blue numbers correspond to key IDs required for each door:



Alice takes keys 0, 1, and 2 while Bob takes keys 3 and 4. Alice walks from 0 to 1, then to 2, and then to 3. There, she drops key 0. She retraces her way back to 1. Bob starts in 1, walks to 4, then to 3, where he picks up key 0. He retraces his way back to 1, and with the newly picked-up key 0, opens up the door to 0.

In the second example, there is no way for both Alice and Bob to reach their destination. Note that Alice cannot drop keys in room 1.

Problem L. Counting Satellites

Time limit	1000 ms
Mem limit	1048576 kB
OS	Linux

Nick likes satellites. He likes them so much that he looks for them everywhere. One day he found a string of letters and counted a lot of instances of the word "`SATELLITE`" among all subsequences of the string. However the next day he forgot this string. Can you help him construct such a string?

String s is a subsequence of string t if and only if it is possible to delete some (possibly zero) characters from t to get s. Two subsequences are considered different if some character at a given position in t is deleted in one subsequence but not the other.

Input

The single line of input contains a single integer k ($1 \le k \le 10^{18}$), which is the number of instances of the word "`SATELLITE`" in the string Nick forgot.

Output

Output a string of at most 5 000 uppercase letters. The string must have exactly k instances of the word "`SATELLITE`" among all its subsequences. It can be proven that under the given constraints a solution always exists. Note that the length of the string does *not* have to be minimized.

Sample 1

Input	Output
1	SATELLITE

Sample 2

Input	Output
2	NICKLIKESSATELLITES

Sample 3

Input	Output
3	SSSATELLITE

Sample 4

Input	Output
19	SATELLITESATELLITE